

DATA AUGMENTATION

- **Types of data augmentation:**
 - ✓ **universal data augmentation**
 - ✓ **task-specific data augmentation**

- **EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks (Wei and Zou, 2019)**
 - **For a given sentence in the training set, we randomly choose and perform one of the following operations:**
 - **Synonym Replacement (SR)**
 - **Random Insertion (RI)**
 - **Random Swap (RS)**
 - **Random Deletion (RD)**

1. **Synonym Replacement (SR)**: Randomly choose n words from the sentence that are not stop words. Replace each of these words with one of its synonyms chosen at random.
2. **Random Insertion (RI)**: Find a random synonym of a random word in the sentence that is not a stop word. Insert that synonym into a random position in the sentence. Do this n times.
3. **Random Swap (RS)**: Randomly choose two words in the sentence and swap their positions. Do this n times.
4. **Random Deletion (RD)**: Randomly remove each word in the sentence with probability p .

- Note that:

Long sentences can absorb more noise while maintaining their original class label because they contain more words than short ones. Therefore, we vary the number of words changed (n) based on the sentence length (l), as below.

- ✓ For SR, RI and RS

$$n = \alpha l$$

- ✓ For RD

$$p = \alpha$$

where:

n = the number of words changed

l = the sentence length

α = a parameter indicating the percent of the words in a sentence that are changed

- **Note that:**
- ✓ For each original sentence, we generate n_{aug} *augmented sentences* (see Table 1, next slide).
- ✓ EDA provided substantial improvements in the case of **small datasets**.
- ✓ For **smaller training sets**, generating many augmented sentences yielded large performance boosts. For **larger training sets**, adding more than **four** augmented sentences per original sentence was unhelpful.

✓ It is best to use parameters in the following way:

N_{train}	α	n_{aug}
500	0.05	16
2,000	0.05	8
5,000	0.1	4
More	0.1	4

Table 1. Recommended usage of parameters:

Stance Detection

- Stance detection differentiates the attitude (e.g. **support**, **oppose**, or **neutral**) of a text towards a topic.
- The topic can be a phrase or a complete sentence.
- The same text can express the author's positions on many different topics.
- Zero-shot stance detection: topics in the test set are unseen during training.

- **Zero-shot learning** is a problem setup in machine learning where, at test time, a learner observes samples from classes which were NOT observed during training, and needs to predict the class that they belong to.
- Zero-shot methods generally work by associating observed and non-observed classes through some form of auxiliary information, which encodes observable distinguishing properties of objects.
- **Few-shot learning (FSL)**, also referred to as low-shot learning (LSL) in few sources, is a type of machine learning method where the training dataset contains limited information, namely:
- **Few-Shot Learning (FSL)** is a Machine Learning framework that enables a pre-trained model to generalize over NEW categories of data (that the pre-trained model has NOT seen during training) using only a few labeled samples per class. It falls under the paradigm of meta-learning (meta-learning means learning to learn).

The Few-Shot Training Scenario

- **Such a setup examines the performance of a model trained with a limited, small number of examples.**

- **Models may either make use of**
 - ✓ **transfer learning****or**
 - ✓ **corpus augmentation techniques****in order to increase the desired performance metric.**

Applications of few-shot learning in NLP:

- **Few-shot learning enables NLP applications to complete tasks with few examples of text data.**
- **These tasks can be listed as:**
 - **parsing**
 - **translation**
 - **sentence completion**
 - **sentiment classification from short reviews**
 - **user intent classification for dialog systems (IBM Research)**
 - **criminal charge prediction (the code used in the study was shared on Github)**
 - **word similarity tasks**
 - **text classification**

A generic method for efficiently augmenting textual corpora using large datasets of unlabeled data

- **The proposed method (Bercaru et al., 2023) is able to efficiently mine for examples similar to the ones that are already present in standard, natural language corpora.**
- **The experimental results (Bercaru et al., 2023) show that using the proposed corpus augmentation methods enables an increase in text classification accuracy in few-shot settings.**
- **Particularly, the gains in accuracy raise up to 16% when the number of labeled examples is very low (e.g., two examples).**

Application on intent classification

- The paper:

Improving Intent Classification Using Unlabeled Data from Large Corpora

(Gabriel Bercaru, Ciprian-Octavian Truica, Costin-Gabriel Chiru and Traian Rebedea, 2023)

<https://www.mdpi.com/2227-7390/11/3/769>

- Intent classification is a central component of a Natural Language Understanding (NLU) pipeline for conversational agents.
- The quality of such a component depends on the quality of the training data.
- For many conversational scenarios, the data might be scarce. In these scenarios, data augmentation techniques are used.

➤ **This recent study (Bercaru et al., 2023) deals with:**

- **exploring the influence of various feature vectors on the task of intent classification using RASA's text classification capabilities;**
- **finding a generic method for efficiently augmenting textual corpora using large datasets of unlabeled data. Specifically:**
- **use unlabeled data to improve the intent detection phase of conversational agents.**

RASA (an open source machine learning framework to automate text and voice-based conversations)

- **RASA** is one of the most successful frameworks for building conversational agents.
- **The RASA architecture** is composed of several interconnected modules, which can function both independently or as a whole.
- **A powerful feature of RASA**: it can integrate state-of-the-art, pre-trained Transformer models, via the Huggingface library (<https://huggingface.co/>)
- These Transformers can increase the intent prediction accuracy, **in the NLU phase**, by providing their own embedding vectors for the supplied tokens.
- **RASA structure** - composed of two loosely coupled sub-systems: the **natural language understanding** (NLU) component and the **dialogue management** component.

- **RASA handles both**
 - simple (query-answer)and
 - complex (multiple turns needed to obtain the required information)conversational scenarios.
- **RASA composed of:**
 - the natural language understanding (NLU) componentand
 - the dialogue management component.
- The intent classification experiments are conducted using only the NLU stage.

The RASA NLU component

- ❖ **Extracts information at a single dialogue turn:**
 - the intent associated with the turn
 - the possible entities in the sentence

- ❖ **The process is divided throughout a pipeline consisting of several stages:**
 - a tokenizer (which splits the raw input text into tokens)
 - one or several featurizers (which encode and extract meaningful information from the tokens)
 - a classification method (which produces the final intent and entities associated with the input sentence).

➤ **Note that:**

- ✓ In the **featurizer stage** multiple methods of encoding the tokens are available.
- ✓ The **encoding mechanism** can employ either ‘standard’ text-based metrics (TF-IDF scores) or embedding vectors obtained through neural models (e.g., word embeddings, Transformer-based models, etc.).

RASA NLU Intent Classification

- **Bercaru et al. (2023) examine the problem of intent classification as part of a conversational agent pipeline.**
- **They define a method for clustering large corpora, to efficiently retrieve examples that are similar to a user-supplied query.**
- **The proposed method is shown to bring statistically relevant improvements in intent classification scenarios in which training data are scarce.**

Datasets Used

- **The performed experiments (Bercaru et al., 2023) use the datasets**
 - ✓ **CLINC150**
 - ✓ **BANKING77**

- **CLINC150** is a dataset for evaluating the performance of **out-of-scope classification systems**. The structure of the main version of the dataset (*full*) is the following:
 - ✓ 150 in-domain classes
 - ✓ one class for out-of-domain examples
 - ✓ each of the 150 domain classes contains 100 training examples, 20 validation examples, and 30 test examples
 - ✓ the out-of-domain class is split into 100 training examples, 100 validation examples, and 1000 test examples.

- **BANKING77:**
 - ✓ contains only queries from the banking domain
 - ✓ these banking queries are divided across 77 in-scope classes
 - ✓ all intents contain the same number of examples (balanced set)

Intent Classification

- One of the simpler featurizers tested is the *CountVectorsFeaturizer* (<https://rasa.com/docs/rasa/components/#countvectorsfeaturizer>)
- *CountVectorsFeaturizer* analyzes the user text and creates a bag-of-words representation based on it. The result is a sparse representation of the input sequence, which disregards token sequentiality.
- Sparse tokenizers are able to extract features at multiple n-gram granularity (standard n values range between 1 and 4), working either at word or character level.
- For evaluation (Bercaru et al., 2023) featurize the text based on character n-grams with sizes between 2 and 4 characters. Note that n-gram extraction is performed on each word's lemma rather than on the original word.

❖ **Remark:**

In order to better capture semantic similarities between words, **dense featurizers** should be used.

➤ **Dense featurizer:** featurizers that produce embedding vectors based on the user utterance.

- **Bercaru et al. (2023) test the following dense featurizers:**

- (1) *SpacyFeaturizer*

- (2) *multiple LanguageModelFeaturizers*

- In the case of *SpacyFeaturizer*, the intent classifier used is *SklearnIntentClassifier* (implemented through Scikit-learn).
- The *LanguageModelFeaturizers* component allows embedding integration mechanisms from state-of-the-art Transformer-based language models.
 - The experiments (Bercaru et al., 2023) employ 6 models:
 - ✓ BERT
 - ✓ ConveRT (a Transformer-based encoder designed for conversations)
 - ✓ RoBERTa
 - ✓ GPT
 - ✓ GPT-2
 - ✓ XLNet

- For all the language model featurizers, the extracted features are used as input for *DIETClassifier*.
- *DIETClassifier* is a multi-task model for intent classification and entity extraction (that uses a single transformer model for both).

For both CLINC150 and BANKING77:

- the training subset is used to fine-tune the models;**
- the test subset is used to compute the accuracy metrics;**
- in all the experiments, the models are fine-tuned for 50 epochs using the training set, except for the SpaCy embeddings setup where fine-tuning is performed for 100 epochs.**

Presentation of results depending on the used classifier

- **The results from Table 2 are obtained without using the Transformer architecture (SklearnIntentClassifier and MitieClassifier).**
- **For all the scores in Table 3, the DIETClassifier was used.**

	CLINC150	BANKING77
SpaCy Embeddings	0.8271	0.8867
CountVectorsFeaturizer	0.7418	0.9026

Table 2. Intent classification accuracy obtained through non-Transformer-based methods. Best performing models for each dataset have their results in bold.

	CLINC150	BANKING77
BERT Embeddings	0.8104	0.9282
ConveRT Embeddings	0.8242	0.9237
RoBERTa Embeddings	0.7651	0.9192
GPT Embeddings	0.7956	0.9081
GPT-2 Embeddings	0.7956	0.9019
XLNet Embeddings	0.7627	0.9006

Table 3. Intent classification accuracy obtained by using different language model features extracted from the user input text. Best performing models for each dataset have their results in bold.

Classification accuracy

The precision, recall, and macro-F1 scores were computed for the language model-based methods (see Table 4).

LMF	CLINC150			BANKING77		
	Recall	Precision	Macro-F1 (avg.)	Recall	Precision	Macro-F1 (avg.)
BERT	0.9455	0.8236	0.8735	0.9282	0.9317	0.9283
ConveRT	0.9475	0.8369	0.8816	0.9237	0.9277	0.9241
RoBERTa	0.9107	0.7856	0.8349	0.9191	0.9220	0.9192
GPT	0.9311	0.8169	0.8614	0.9081	0.9122	0.9084
GPT-2	0.9066	0.7868	0.8332	0.9019	0.9056	0.9018
XLNet	0.8986	0.7813	0.8272	0.9006	0.9052	0.9003

Table 4. Recall, precision, and F1 classification scores obtained using different types of language model featurizers (LMF). Best performing models, in terms of macro-F1 average score, have their results in bold.

➤ **Note that:**

Both in Table 3 and in Table 4 the highest F1 scores are obtained by using the ConveRT (for CLINC150) and BERT (for BANKING77) language model featurizers.

CORPUS CLUSTERING

The Method

- **This method (Bercaru et al., 2023) can be regarded as a pipeline which:**
 - ✓ **processes raw transcripts;**
 - ✓ **embeds individual sentences in order to obtain dense feature representations;**
 - ✓ **clusters them in order to shorten similar sentence retrieval time.**

The Method

1. Data Preprocessing

- **The proposed system uses the subset of English subtitles from the OpenSubtitles corpus as the training set.**
- **The subtitles are encoded as XML files.**
- **For this set of experiments, only the raw text of the subtitles was used.**
- **The initial set, including time annotations, contains 123 GB of data split across approximately 446,000 subtitle files.**
- **The final corpus [see data preprocessing details in (Bercaru et al., 2023)] is reduced to only 140,000 subtitle files, or 4 GB of raw text, split into 37 record files, approximately 100 MB of data each.**
- **The number of sentences per record remains unchanged.**
- **The total number of examples available is lowered to 131 million.**

2. Embedding, Clustering and Data Projection

- Following the success of the Transformer architecture in numerous NLP tasks, several types of embedding vectors obtained through the encoder modules of different Transformer models may be used.
 - Here the chosen ones are:
 - ✓ A BERT model from Huggingface
 - ✓ An SBERT (Sentence BERT) model
- Online clustering algorithms can be used to cluster sentences.
 - One such example is *mini-batch K-means*. Similar to standard K-means, it optimizes the same non-convex objective function, while iteratively processing batches of the input data **X**.

- In the current implementation (Bercaru et al., 2023), fitting the data through mini-batch K-means is done for a fixed number of steps rather than until a given convergence criterion is met.
- After fitting the current examples, the embedding vectors and corresponding cluster labels are stored on the disk, to allow the processing of the next batch of embedding vectors.
- Having the instances from the learning set embedded and clustered, performing inference for a given test instance q consists of the steps described in Algorithm1.

Algorithm 1 Inference steps for a given test example q

Input: E embedder, q inference sentence

Output: p the <sentence, index> pair

1: $\mathbf{emb}_q \leftarrow E(q)$

▷ Obtain the embedding vector

2: $c_q \leftarrow$ closest cluster to $\mathbf{emb}_q$

▷ Identify the closest cluster

3: $m_q \leftarrow$ closest example to $\mathbf{emb}_q$ in c_q

4: $p \leftarrow \text{index}(m_q)$

▷ Index in the original corpus

5: **Return** p

The steps of Algorithm 1 explained:

- Obtain the embedding vector emb_q for q based on the embedder E used for performing clustering (Line 1).
- Identify the closest cluster center c_q to emb_q , as computed through mini-batch K-means, i.e., compute the cosine similarity against all K clusters and return the most similar cluster center (Line2).
- Identify the closest embedding vector to emb_q , in the current cluster c_q , i.e., compute the cosine similarity against all learning examples assigned to cluster c_q , and return the most similar example (Line 3).
- Lookup in the original corpus and retrieve p the natural language sentence paired with the index based on the embedding vector index (Lines 4 and 5).

Remark (data augmentation):

Not all of the sentences retrieved through the described method would positively impact the quality of the corpus which is to be augmented. However, given a limited set of hand-chosen learning examples, one can train a weak classifier with the initial set of examples. The additional examples retrieved by the system can be filtered based on the classifier class output probabilities, i.e., if an example is assigned to a specific class with a probability greater than a threshold, then the example will be further considered for augmenting the corpus; otherwise, the low probability will lead to the dismissal of the example.

Evaluation and Results

- **To evaluate the proposed similar sentence retrieval system, Bercaru et al. (2023) use it to augment the intent classification training sets, after which they evaluate the models trained with the augmented data on the unmodified test sets, using RASA's DIETClassifier.**

➤ **Evaluation scenarios:**

- **The first scenario aimed to verify to what extent the number of clusters impacts the classification accuracy.**
 - ✓ **Bercaru et al. (2023) experiment with 512 and 1024 clusters.**
 - ✓ **In both setups, the complete training sets of BANKING77 and CLINC150 are used to train initial DIETClassifier models.**
 - ✓ **These classifiers are subsequently used for classifying the additionally extracted examples.**
 - ✓ **Only examples classified with at least 0.9 confidence are used for augmenting the training sets.**

- **The second scenario:**

The second set of experiments is conducted to evaluate the sentence retrieval system in a **few-shot scenario**:

- ✓ the training sets of BANKING77 and CLINC150 are sequentially restricted to only $k \in \{2, 3, 5, 10\}$ examples per class.
- ✓ The corpus clustering method is then used to artificially increase the number of examples available, based on the initial, limited number of examples. For each initial example x_i , a similar sentence y_i is retrieved.
- ✓ All retrieved y_i are aggregated and combined with the initial learning set with the duplicates being removed.
- ✓ Evaluation of the few-shot setups is performed by training an ensemble of 10 different DIETClassifier models in each configuration.

- **Typical of few-shot scenarios: minimizing the number of noisy examples which are added to the learning set**
 - **The following heuristic is used (Bercaru et al., 2023):**
 - ✓ **When attempting to add a retrieved candidate y_i to a class C , first compute its set of unique tokens.**
 - ✓ **Stop words are removed before the above set computation.**
 - ✓ **Perform a set intersection with the set of tokens corresponding to all initial examples x_i in the class C .**
 - ✓ **Only add the example if the set intersection size exceeds a given threshold $t \in \{0, 1, 2\}$.**

DISCUSSION concerning the few-shot scenario:

- **For the second set of experiments regarding the few-shot scenario, Bercaru et al. (2023) restrict the training sets of BANKING77 and CLINC150 to only 2, 3, 5, or 10 examples per intent.**
- **The corpus clustering method is then used to artificially increase the training set sizes. In the best-case scenario, the sizes are doubled.**
- **Duplicate similar examples that are retrieved are removed.**
- **Additional examples are removed according to the described heuristic.**
- **In this setup, the proposed corpus clustering method helps improve the classification accuracy, in the best case leading to 16% accuracy increase for CLINC150 when $k = 2$ and a 14% accuracy increase for BANKING77 when $k = 2$.**
- **As more of the original training examples become available, the proposed method still increases the mean classification accuracy, but to a smaller extent.**

Final Conclusions

- **Bercaru et al. (2023) examine the problem of intent classification as part of a conversational agent pipeline.**
- **They define a method for clustering large corpora, to efficiently retrieve examples that are similar to a user-supplied query.**
- **The corpus clustering method is shown to bring minor improvements in terms of classification accuracy when the full training sets are available.**
- **The authors also examine to what extent the method helps improve the accuracy when a limited number of examples are available.**
- **The results have shown that the intent classification accuracy is raised by up to 16%, in the most favorable case, where only two labeled examples per class are available.**

- **The proposed method (Bercaru et al., 2023) achieves retrieval times as low as 0.2–0.3 s per example and is shown to bring statistically relevant improvements in intent classification scenarios in which training data are scarce.**
- **For tasks in which large datasets are available, the method does not introduce significant improvements. However, for small datasets, it helps improve the diversity of the examples, thus leading to significantly larger accuracy scores.**